

LLM Notes

2026 Spring

Day 15: 训练步骤的时间构成与硬件瓶颈

CS336 Systems

主讲: Aaron

Fudan University

今日摘要

2026.7.17 一次基本训练步骤可以拆成 forward、backward 和 optimizer update。Transformer 的 forward/backward 主要由矩阵乘法构成, 通常更偏 compute-bound; AdamW 每个参数计算量较少, 但需要大量读写参数和优化器状态, 通常更偏 memory-bandwidth-bound。

各阶段的时间比例不是固定常数。Forward 和 backward 会随 batch size 与 sequence length 改变, 而 AdamW 主要随参数量改变, 因此短序列下 optimizer 可能占据很高的训练时间比例。

15.1 训练步骤的组成

不考虑 gradient clipping、数据加载和通信时, 一次训练步骤近似为:

$$T_{\text{train}} = T_{\text{forward}} + T_{\text{backward}} + T_{\text{optimizer}}.$$

15.2 实验结果

在 FP32、batch size $B = 4$ 、sequence length $S = 512$ 的实验中, 时间分解如下:

模型	Forward/ms	Backward/ms	AdamW/ms	Train Step/ms
Small	20.06	41.31	7.72	69.09
Medium	55.11	113.52	24.89	193.52
Large	123.68	259.20	52.02	434.89
XL	334.58	732.38	165.24	1232.20

15.2.1 Forward 与 Backward

由实验得到：

模型	Backward / Forward
Small	$2.06\times$
Medium	$2.06\times$
Large	$2.10\times$
XL	$2.19\times$

因此，在本实验中可以近似写成：

$$T_{\text{backward}} \approx 2T_{\text{forward}}.$$

对于一次 Linear 或矩阵乘法，前向需要计算输出；反向通常还需要计算：

$$\frac{\partial L}{\partial X} \quad \text{和} \quad \frac{\partial L}{\partial W}.$$

也就是输入梯度和参数梯度，所以主要矩阵乘法的反向计算量大约是前向的两倍。

15.3 AdamW 的时间占比

实验中 AdamW 相对于 backward 的占比为：

模型	AdamW / Backward	AdamW / Train Step
Small	18.7%	11.2%
Medium	21.9%	12.9%
Large	20.1%	12.0%
XL	22.6%	13.4%

对于上述 $S = 512$ 的实验，可以得到经验近似：

$$T_{\text{AdamW}} \approx 0.2T_{\text{backward}},$$

$$T_{\text{train}} \approx 3.2T_{\text{forward}}.$$

对于结构已经确定的模型，改变 B 或 S 不会改变参数数量；它们改变的是 token 数量、activation 大小以及 forward/backward 的计算量。

15.4 Compute-bound 与 Memory-bandwidth-bound

GPU 执行一个算子时，主要可能受到两种能力限制：

定义

定义 15.4.1 (Compute-bound). 算子需要执行大量浮点运算，运行时间主要受 GPU 计算单元吞吐量限制。即使数据已经到达计算单元，GPU 仍需要花费较长时间进行计算。

定义

定义 15.4.2 (Memory-bandwidth-bound). 算子每读取一个数据只进行少量计算，运行时间主要受显存与计算单元之间的数据传输速度限制。计算单元可能在等待数据，而没有被充分利用。

判断一个算子更接近哪种情况，可以考虑 arithmetic intensity：

$$I = \frac{\text{FLOPs}}{\text{从显存读写的字节数}}.$$

Arithmetic intensity 高的算子更容易 compute-bound；反之更容易 memory-bandwidth-bound。

15.5 为什么矩阵乘法偏 Compute-bound

对于矩阵乘法

$$C = AB,$$

若

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N},$$

其计算量约为：

$$2MKN \text{ FLOPs}.$$

矩阵元素从显存读入后，可以在分块矩阵乘法中被重复使用。随着矩阵规模增大，计算量的增长通常快于必要的数据搬运量，因此 arithmetic intensity 较高。

Transformer 中的 Linear、QK matmul、AV matmul 和 MLP matmul 都具有这一特点，所以 forward 和 backward 通常主要消耗 GPU 的矩阵计算能力。

15.6 为什么 AdamW 偏 Memory-bandwidth-bound

AdamW 对每个参数执行类似下面的更新：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t,$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{m_t}{\sqrt{v_t} + \epsilon} + \lambda \theta_{t-1} \right).$$

每个参数只执行较少的逐元素运算，但需要访问多个大型 tensor：

- 参数 θ ;
- 梯度 g ;
- 一阶矩 m ;
- 二阶矩 v 。

以 FP32 为例，仅考虑必要的主要读写，每个参数通常至少涉及：

$$\underbrace{4}_{\text{读取梯度}} + \underbrace{8}_{\text{读写参数}} + \underbrace{8}_{\text{读写一阶矩}} + \underbrace{8}_{\text{读写二阶矩}} \approx 28 \text{ bytes}.$$

实际流量会受到 fused optimizer、缓存、临时 tensor 和具体实现影响，因此不应简单固定为某个精确字节数。

分析

AdamW 的问题不是计算公式特别复杂，而是模型的每个参数、梯度以及两个 optimizer states 都需要被访问。每读取大量数据只进行少量逐元素运算，因此 arithmetic intensity 很低，运行时间通常更受显存带宽限制。